Chapter 6

Self-Organizing Maps (SOM)

Draft: do not distribute

A motivation for reducing the dimensionality of the data is to make it possible to visualize high-dimensional data, e.g., by projecting them onto a suitable lower dimensional subspace. Successful data reduction and visualization by projection methods often require that an orthogonal projection onto a linear subspace captures salient features of the data, such as the presence of clusters. It is easy to find examples where the data are supported on or near a low-dimensional manifold, yet the intrinsic low dimensionality and the organization of the data are not correctly represented by any orthogonal projection onto a linear subspace. This is the case, for instance, when the data are distributed on or near the surface of a sphere in $\mathbb{R}^3$; in fact, while the surface is clearly two-dimensional, a projection onto a two-dimensional plane may confound the organization of the data, since any pair of antipodal points along a ray orthogonal to the plane will be represented as being the same point, while when measured along the surface of the sphere, the points are distant.

A *Self-Organizing Map (SOM)* algorithm creates a low-dimensional prototype-based discrete representation of the data. An SOM algorithm approximates the data using relatively few prototype vectors that are topologically organized in a particular way. The algorithm keeps track of the low dimensional topological structure of the prototypes relative to each other as they organize themselves to approximate the distribution of the data. In this manner, each prototype links the nearby data points to a specific position in a lower dimensional lattice, thus creating a map of the data suitable for visualization.

Self-organizing maps belong to the vast class of artificial *neural networks*, since they are based on combinations of adaptation and learning. In the following, we first provide some intuition of how SOM algorithm works, then discuss how one could set the parameters so as to render the method functional in a variety of applications. Since the algorithm does not rely on a solid mathematical foundation, there are no guarantees that the prototypes will successfully capture the organization of the data, and some of the algorithmic choices are based on heuristic considerations and extensive numerical testing.

6.1 SOM: the basic ideas

The inspiration for the original SOM algorithm is an idealization of how the brain is adaptively organizing itself to process external stimuli. The description involves the concepts of geometry and topology, interpreted as neighborhood structure. The original description of the SOM algorithm recall how brain regions that process similar stimuli become organized to be topologically near each other, in the sense that they form strong connections regardless of their actual location. The stimuli, on the other hand, bear varying degrees of similarity that can be captured in geometric terms. Within the SOM framework, the data represent the stimuli, and the topological structure is encoded in a low dimensional feature map with a simple topological organization. The SOM algorithm is an adaptive learning scheme in which geometric nearness in a high-dimensional space is translated into topological nearness in a low-dimensional feature map.

For the sake of clarity, we assume that the data lives in n-dimensional Euclidian space $\mathbb{R}^n$, and is viewed as a stream of vectors $x^{(t)} \in \mathbb{R}^n$, $t = 1, 2, \ldots$. Further, we assume that either the data vectors $x^{(t)}$ are distributed according to an unknown probability distribution in $\mathbb{R}^n$, or, if the data are a finite set of observations organized in a matrix

$$\mathsf{X} = \begin{bmatrix} x^{(1)} & x^{(2)} & \cdots & x^{(p)} \end{bmatrix} \in \mathbb{R}^{n \times p},$$

the vectors $x^{(t)}$ are generated by drawing columns of X with replacement in a random order. The goal of the SOM algorithm is to explore and visualize the distribution of the data vectors in the space $\mathbb{R}^n$.

To provide some intuition about SOM, consider the setting in Figure 6.1, where the data are points in a two-dimensional space distributed in the vicinity of a curve, hence inherently one-dimensional, although there is no immediate way of capturing the organization of the data by projecting onto a line. Instead, we seek to discover the one dimensional organization of the data with the help of K prototype vectors m_ℓ, $1 \leq \ell \leq K$ in the data space $\mathbb{R}^2$. To keep track of the topology of the data set, we define a neighborhood structure for the prototypes by introducing a one-dimensional lattice Q, often referred to as feature map, with lattice points q_ℓ such that two prototype vectors m_ℓ and m_k are neighbors if and only if the corresponding lattice points q_ℓ and q_k are neighbors. Formally, we may think that there is a continuous map

$$\mathbb{R} \to \mathbb{R}^2, \text{ such that } q_\ell \mapsto m_\ell,$$

inducing a neighborhood structure among the prototype vectors m_ℓ. In this manner, the prototypes can be thought of as points on a low-dimensional manifold fitted to the data, and the lattice points q_ℓ are discrete points in the low-dimensional coordinate space.

In principle, the lattice space $Q = \{q_1, q_2, \ldots, q_K\}$ can be of any dimensions, although generally one- or two-dimensional lattices are preferred. In the one-dimensional case, a linear lattice is normally used, that is,

$$q_\ell = \ell, \quad 1 \leq \ell \leq K.$$

Draft: do not distribute

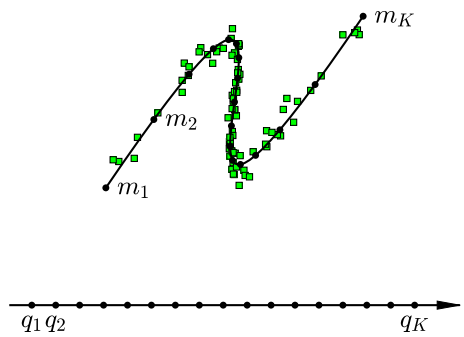

Figure 6.1. *Intuitive idea of SOM. Given the data vectors in $\mathbb{R}^2$, represented as green squares, we want to explore their distribution. The SOM algorithm seeks to fit a discrete set of prototype vectors m_ℓ in the data space, with a low-dimensional neighborhood structure inherited from the discrete lattice $Q = \{q_1, q_2, \ldots, q_K\}$. Therefore the prototypes can be thought of as points on a low dimensional manifold fitted to the data.*

Draft: do not distribute

In two dimensions, popular choices include triangular, rectangular or hexagonal lattice structures. For a rectangular lattice of size $N \times N$, we may number the lattice points by setting

$$q_\ell = (i, j), \quad \ell = N(i-1) + j, \quad 1 \le i, j \le N, \tag{6.1}$$

so that $1 \le \ell \le K = N^2$, and introduce the natural neighborhood structure in the Euclidian space $\mathbb{R}^2$. In correspondence of each lattice point q_ℓ, we define a prototype vector $m_\ell \in \mathbb{R}^n$. The SOM algorithm can be summarized as follows.

1. Start by defining K prototypes m_ℓ, representing grid points on a parametrized low-dimensional manifold, such as a curve or surface, in the data space.

2. When presented with a data vectors $x^{(t)}$, the prototypes move in a concerted way towards the data, respecting their underlying topology. When the position of one prototype is adjusted to better capture the data, its topological neighbors are adjusted as well. The neighborhood structure is determined by the underlying lattice topology

In the following section, we discuss some details of the learning process that are essential to make the SOM algorithm robust and versatile.

6.2 Learning the structure of the data

We begin by introducing a low dimensional lattice $Q = \{q_1, q_2, \ldots, q_K\}$ and assigning to each lattice point a prototype vector $m_\ell \in \mathbb{R}^n$. The initial configuration of the prototypes can be random, with the only requirement that each prototype vector is unique, $q_\ell \neq q_k$ for $\ell \neq k$.

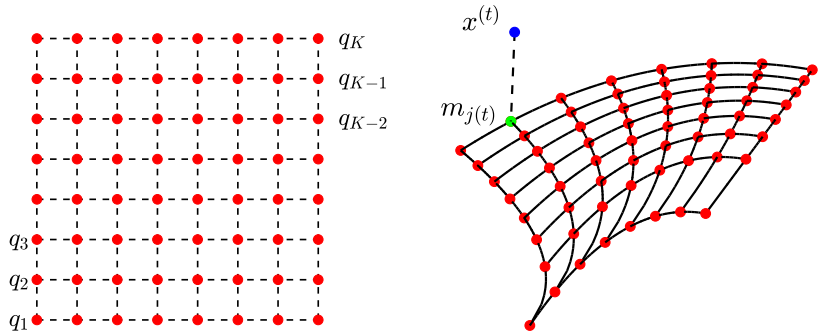

Figure 6.2. *Left panel: A rectangular grid of lattice points enumerated column by column as given by formula (6.1). Right panel: The corresponding feature vectors, with the topology indicated by connecting neighboring points to form a fictitious mesh. The BMU $m_{j(t)}$ corresponding to the data vector $x^{(t)}$ is indicated by green.*

Draft, do not distribute

Consider a discrete time $t = 1, 2, \ldots$, and at each time instance, select a data vector $x^{(t)}$ and identify its *best matching unit (BMU)*, that is the nearest prototype vector. If we use the Euclidian distance in the data space, the BMU is the prototype $m_{j(t)}$ such that

$$\|m_{j(t)} - x^{(t)}\| = \min\{\|m_\ell - x^{(j)}\| \mid 1 \le \ell \le K\},$$

where $j(t) \in \{1, 2, \ldots, K\}$ is the index pointing to the closest prototype, or BMU. This is illustrated schematically in Figure 6.2.

Once the BMU of the data point $x^{(t)}$ has been determined, we move it towards the data point by updating its position according to the formula

$$m_{j(t)} \to m_{j(t)} + \alpha(x^{(t)} - m_{j(t)}), \tag{6.2}$$

where the coefficient α is the *learning rate*, $0 < \alpha < 1$. If $\alpha = 1$, the updated prototype coincides with $x^{(t)}$.

If only the BMU is updated at each time instance and the process is repeated with a sequence of several data points, the information about the mutual relations between prototypes is quickly lost. On the other hand, we would like to allow the distance between prototypes to stretch, since stretching does not change the topology, but only the geometry. We resolve the problem of adjusting the prototypes according to the organization of the data without losing track of their topological organization by measuring their nearness in the lattice space Q, rather than in data space $\mathbb{R}^n$. Therefore we state that

$$m_{\ell_1} \text{ and } m_{\ell_2} \text{ are near each other} \Leftrightarrow q_{\ell_1} \text{ and } q_{\ell_2} \text{ are near in } Q.$$

The nearness of the lattice points may be measure in any metric. Using the Euclidian metric to measure the distance between the lattice points, we define the distance

matrix $\mathsf{D} \in \mathbb{R}^{K\times K}$ with entries

$$d_{ij} = \|q_i - q_j\|, \quad 1 \le i, j \le K.$$

The idea in the SOM learning phase is to update not only the BMU, but also prototypes topologically near it. We quantify how tightly a prototype is tied to other prototypes by introducing a *neighborhood matrix* $\mathsf{H} \in \mathbb{R}^{K\times K}$, measuring the coupling strength. A commonly used neighborhood matrix can be constructed by defining

$$h_{ij} = \exp\left(-\frac{1}{2\gamma^2} d_{ij}^2\right), \quad 1 \le i, j \le K,$$

where $\gamma > 0$ is a coupling constant. The smaller the value of the coupling constant γ, the narrower the region of influence around each lattice point.

To retain the neighborhood information in the prototype updating step, we extend the updating scheme (6.2) to all prototype vectors. More specifically, for each ℓ, we update the corresponding prototypes according to

$$m_\ell \to m_\ell + \alpha h_{j(t),\ell}(x^{(t)} - m_\ell). \tag{6.3}$$

In this manner, the BMU m_ℓ is updated as in (6.2) since $h_{j(t),j(t)} = 1$ and, in addition, the closer the lattice point q_ℓ is to $q_{t(j)}$, the more the prototype vector m_ℓ is updated, while prototypes associated with distant lattice coordinates remain essentially unchanged. We may think of the prototype vectors as being connected to each others through an elastic mesh, with the elasticity of the links between nodes determined by the neighborhood structure in Q. The effect of the coupling constant on the updating is explained schematically in Figure 6.3.

Draft: do not distribute

6.2.1 Dynamic refinement

It is common that, in order for dynamic adaptive schemes to give good results, there should be a fast and robust learning phase in the beginning, following by a more conservative tuning phase. In the SOM algorithm, the two factors that should be changing over time are the learning rate and the coupling constant in the neighborhood function.

A way to have the prototypes adapt to the data set is to make the learning rate α a function of time t. Extensive numerical tests reported in the literature suggest that the learning rate should be close to one at first, gradually decreasing close to zero during the learning phase, then remain constant at the small value during the tuning phase, with the functional relation not playing a big role. A suitable expression for the learning rate is

$$\alpha(t) = \max\{\alpha_0(1 - t/T_0), \alpha_1\}, \tag{6.4}$$

for $1 > \alpha_0 > \alpha_1 > 0$, where α_0 is the initial learning rate, α_1 the final one, and T_0 is a time constant controlling the duration of the *learning phase.* When $t > T_0$, the algorithm is in the *tuning phase*, and we have $\alpha = \alpha_1$.

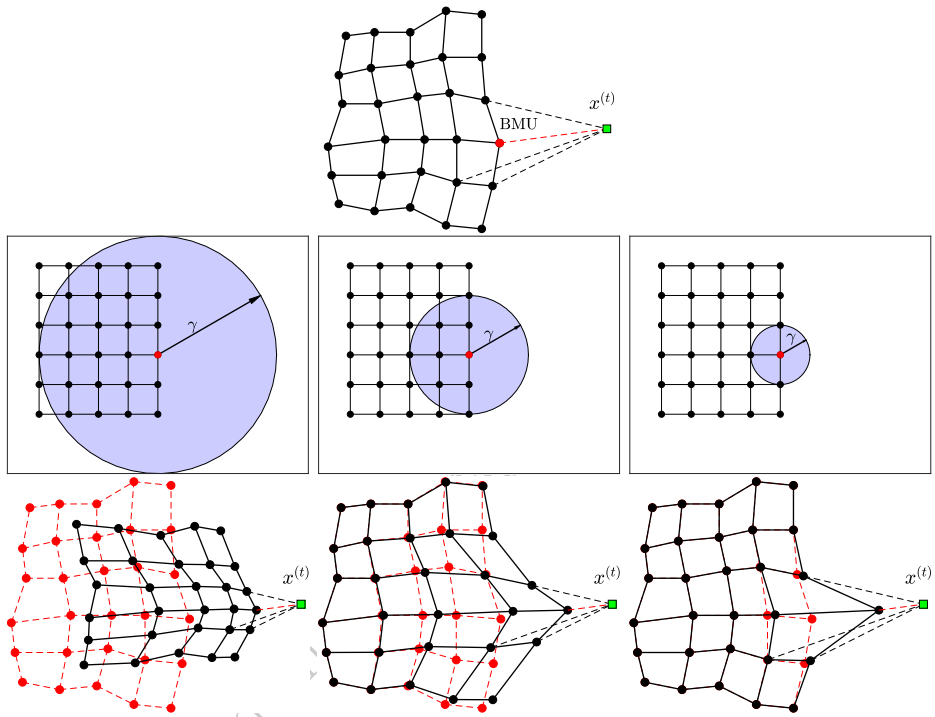

Figure 6.3. *Effect of the coupling constant. The top row shows the initial configuration of the prototype vectors, connected to each other according to the topology. The BMU closest to the current data $x^{(t)}$ is indicated. In the second row, the blue disc indicates the domain of influence corresponding to different values of the coupling constant. The bottom row shows the updates of the prototypes based on the neighborhood matrix with the decreasing value of the coupling constant. When γ is large, the prototypes are strongly coupled, and several of them move along with the BMU, while for small γ, the BMU affects the prototypes only locally, and only the closest prototypes move.*

Moreover, at the beginning of the learning phase it is important that the algorithm adjusts the positions of a much larger portion of prototype vectors rather than the BMU and its immediate neighbors, as illustrated in Figure 6.3, progressively reducing the number of prototypes undergoing adjustment to the BMU and its immediate neighbors. This can be achieved by making the coupling constant γ in the neighborhood function time dependent. Following what was done for the learning rate, we propose a piecewise linear model for γ of the form

$$\gamma(t) = \max\{\gamma_0(1 - t/T_0), \gamma_1\}, \tag{6.5}$$

where $\gamma_0 > \gamma_1 > 0$ are the parameters determining the width of the neighborhood

function during and after the learning period. Then

$$h_{ij}(t) = \exp\left(-\frac{1}{2\gamma(t)^2} d_{ij}^2\right), \quad 1 \le i, j \le K.$$

We are now ready to present the details of the SOM algorithm.

SOM algorithm

---

1. **Given** k, the number of prototypes, and a low-dimensional lattice $Q = \{q_1, \ldots, q_K\}$, equipped with a neighborhood structure, or topology. Set $T_{\max}$ to be the maximum number of iterations.
2. **Initialize:** Define the initial prototypes m_ℓ, $1 \le \ell \le K$, $m_j \ne m_k$ for $j \ne k$. Set $t = 0$. Denote the initial prototypes by $m_\ell(0)$.
3. **Iteration:** for $t < T_{\max}$

   Draw a data vector $x^{(t)}$,

   Find the best matching unit (BMU) $m_{j(t)}(t)$ among the current prototypes $\{m_\ell(t), 1 \le \ell \le K\}$,

   $$\|m_{j(t)}(t) - x^{(t)}\| = \min\{\|m_\ell(t) - x^{(t)}\| \mid 1 \le \ell \le K\},$$

   Compute the current learning rate $\alpha(t)$ and the $j(t)$th column of the neighborhood matrix $h_{ij}(t)$, and update all prototype vectors,

   $$m_\ell(t+1) = m_\ell(t) + \alpha(t) h_{\ell,j(t)}(t)\big(x^{(t)} - m_\ell(t)\big).$$

   Advance the counter by one, $t \to t + 1$.

We conclude this section with a brief discussion ont how to choose the values of the parameters in the SOM algorithm.

(a) The total time span $T_{\max}$ is a critical parameter whose value must be large enough to obtain meaningful results. It is suggested in the literature to set $T_{\max} \ge 500 \times K$, where K is the number of prototypes.

(b) The length of the learning phase must also be long enough. Again, a suggested value in the literature, based on extensive testing, is $T_0 > 1\,000$.

(c) The learning rate in the beginning should be close to one, and $\alpha_0 = 0.9$ seems to be a good choice. At the end of the learning phase, the learning rate should stabilize to a low value, e.g., $\alpha_1 = 0.01$ or smaller. This is the value at which the learning rate is kept during the tuning phase.

Draft: do not distribute

(c) The coupling constant should be large in the beginning, so that the value of the neighborhood function is large in a disc of influence around the lattice point related to the BMU with radius at least one third of the width of the lattice. If the lattice has width N, setting $\gamma_0 = N/3$ is a reasonable choice. At the end of the learning phase, the disc should comprise only few of the neighbors of the BMU lattice points, thus setting $\gamma_1 = 0.5$ is reasonable.

In the discussion above, it was implicitly assumed that the lattice points are roughly at a unit distance from each other. If this is not the case, the values for γ_0 and γ_1 should be scaled accordingly.

The following section illustrates some features of the SOM algorithm by means of computed examples that will show also some novel uses of the SOM .

6.3 Computed examples

Draft: do not distribute

We begin with a simple example where the lattice is one-dimensional. We then present a two-dimensional lattice example, and show how the SOM algorithm can be used to visualize high dimensional data. Finally, we apply the algorithm to real data sets.

6.3.1 One-dimensional coordinate lattice

In this example we assume that data arrives as a discrete stream of vectors $x^{(t)}$, $t = 1, 2, 3, \ldots$, and that the SOM algorithm learns the distribution of the data points in a sequential manner. To illustrate this point, we generate the data vectors $x^{(t)} \in \mathbb{R}^2$, $t = 1, 2, \ldots$ by randomly drawing an angle $\theta^{(t)} \in [0, 2\pi]$ from the uniform distribution over the interval, and define the corresponding data point $x^{(t)} \in \mathbb{R}^2$ as

$$x^{(t)} = \begin{bmatrix} \cos\theta^{(t)} \\ \sin\theta^{(t)} \end{bmatrix}.$$

This is tantamount to generating data vectors $x^{(t)}$ uniformly distributed on the unit circle. Ideally the SOM prototype vectors should organize themselves uniformly on the unit circle.

We set the number of prototypes to $K = 30$, and initialize the prototype vectors by drawing them randomly from the uniform distribution over the square $[0, 0.1] \times [0, 0.1]$. Following the guidelines at the end of the previous section, we set $T_{\max} = 500 \times K = 15\,000$ and t $T_0 = 5\,000$.

The topological lattice space Q in this example consists of integers, $Q = \{1, 2, \ldots, K\}$. The parameters for the learning rate and the coupling constant are

$$\alpha_0 = 0.9, \quad \alpha_1 = 0.01, \quad \gamma_0 = K/3, \quad \gamma_1 = 0.01,$$

with a Gaussian neighborhood function. Figure 6.4 shows twelve snapshots of the evolution of the prototype vectors with the underlying neighboring lattice links represented by a line segment. We observe that at the beginning of the learning phase, the prototypes move significantly because of the high learning rate and move

as a group due to the large coupling parameter. As the learning rate becomes smaller, the movements decrease, and with weaker coupling, the prototypes become more independent. At the end, the prototype distribution is close to uniform along the unit circle, as expected, capturing well the organization of the data.

Once the prototypes have learned the structure of the data, we can think of them as a sort of feature vectors and find different uses for them. For example, we may partition an independent sequence of data, $\widehat{x}^{(1)}, \widehat{x}^{(2)}, \ldots, \widehat{x}^{(q)}$ into K parcels according to the nearest prototype and plot a histogram over the lattice coordinates. In this manner, the SOM naturally defined a coarse map that reduces the data dimensionality.

6.3.2 Two-dimensional lattice

In the second example, the data are 300 points in $\mathbb{R}^3$, scattered around the surface of a hemisphere of the unit ball and partitioned into three clusters, each with 100 points. Since the data lie near the spherical surface, we can hope that the SOM algorithm will be able to approximate the surface with the prototypes, and provides a way to represent the data in a flat surface. We choose a rectangular coordinate lattice of cardinality $K = 100$,

$$Q = \{(i, j) \mid 1 \leq i, j \leq 10\}$$

and initialize the prototype vectors by randomly drawing 100 points from the uniform distribution over a small cube $[0, 0.1]^3$. We set $T_0 = 5\,000$ and $T = 300 \times K = 30\,000$. The parameters in the learning rate are chosen as in the previous example,

$$\alpha_0 = 0.9, \quad \alpha_1 = 0.01,$$

while setting the coupling parameter to

$$\gamma_0 = 10/3, \quad \gamma_1 = 0.01,$$

that is, at the beginning γ is one third of the width of the lattice in the coordinate directions.

Figure 6.5 shows two different views of the final approximation of the data set by the prototypes. In the panels, prototypes whose lattice points are horizontal or vertical neighbors are connected by a line segment, creating a fictitious mesh that helps understand the self-organizing nature of the outcome.

We conclude this section by showing how to use the SOM prototypes to represent higher-dimensional data in a low-dimensional scatter plot. We associate to each data point $x^{(j)}$ its nearest prototype, or BMU, thus subdividing the data into K parcels. Since each prototype corresponds to a lattice point g_ℓ, we draw a small disc around each lattice point, and draw a dot in that disc for each data point in the corresponding parcel, using colors to reflect the annotation of the data. To make the visual representation more compelling, we position each dot randomly in that disc. Figure 6.6 shows the resulting "button plot" of the data. This plot captures important features of the organization of the data, for instance the fact that the

Draft: do not distribute

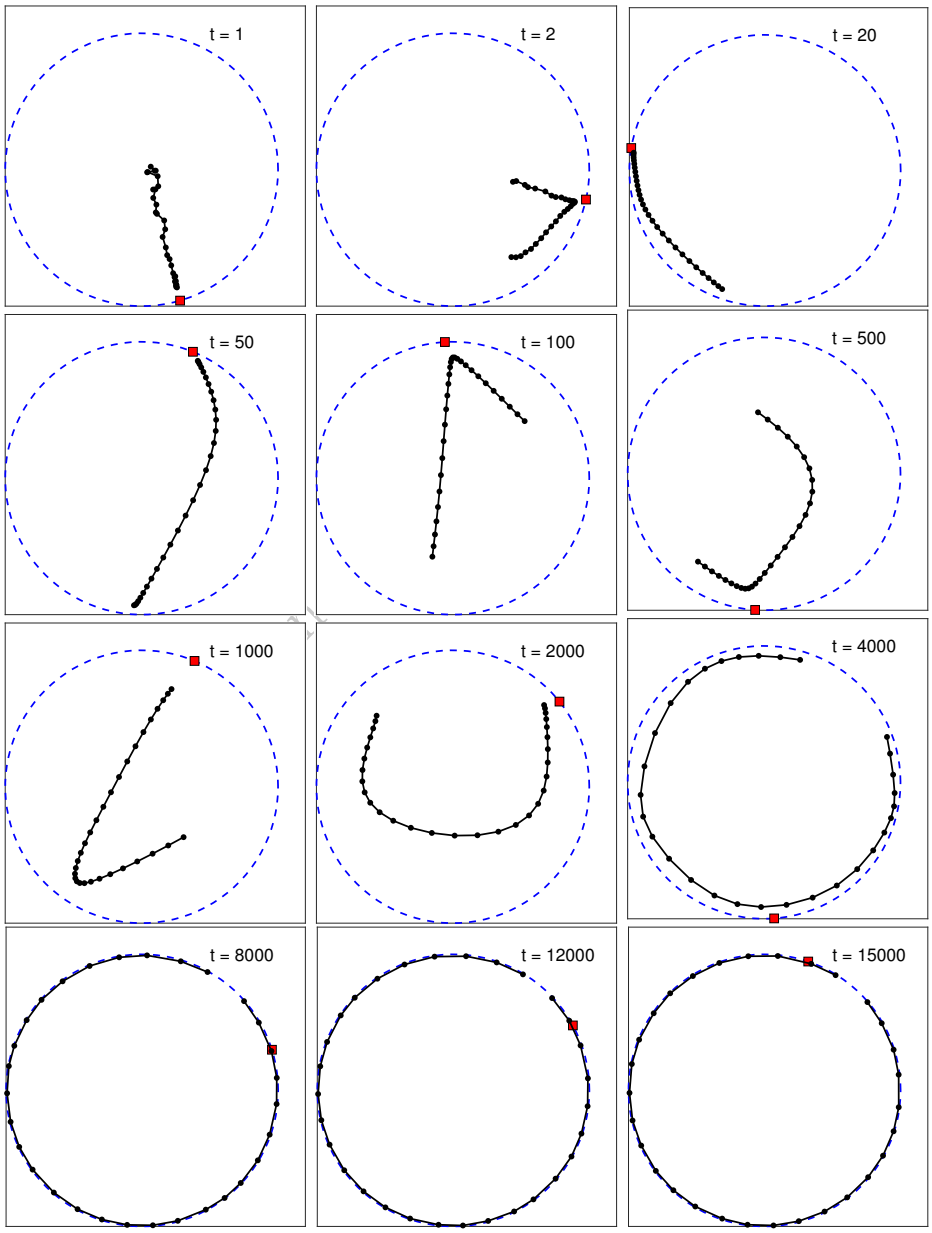

Figure 6.4. *A sequence of time instances of the adaptation process of the SOM prototypes. In each panel, the pertinent data vector* $x^{(t)}$ *is indicated by a red square, and the prototypes* m_ℓ *are joined with a fictitious curve indicating the one-dimensional neighborhood relation.*

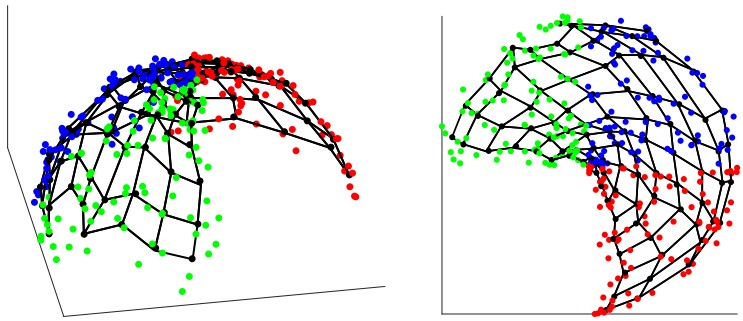

Figure 6.5. *Two different views of the three dimensional data set, with each data point being marked with a color indicating its annotation, and the feature vectors marked as black dots. The adjacency of the underlying lattice coordinates is indicated with a segment connecting the feature vectors, thus creating an artificial mesh surface.*

Draft: do not distribute

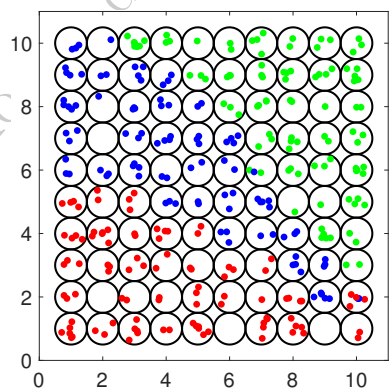

Figure 6.6. *Representation of the three-dimensional data shown in Figure 6.5 in a two-dimensional map. The data are divided in 100 parcels according to the closest prototype, and in correspondence of each vector in a parcel, we draw a dot inside a small disc around the corresponding lattice points using the color encoding the annotation. The positions of the colored dots in the discs are chosen at random to avoid that the dots fall on top of each other.*

three subgroups are neatly separated, and that the red and blue groups, as well as blue and green groups are adjacent to each other.

While the annotation in this example was arbitrary, the SOM algorithm has been successfully used to identify hidden similarities between high dimensional data vectors, observing that when represented in the lattice space, similar data vectors end up near each other.

6.3.3 Hyperspectral imaging

Hyperspectral imaging is a remote sensing modality that uses multi-frequency data from a ground target. The data collection consists of an airborne infrared/near-infrared camera recording the upwelling radiance from a terrain target over a band of wavelengths. Each pixel in the image, representing a certain portion of the ground, has a characteristic spectrum that depends on the properties and conditions of the ground area that determine the radiance. Vegetation canopy radiates differently from a constructed area or body of water, and different vegetation types produce different spectra. The radiance changes also due to weather conditions such as sunshine and precipitation. Therefore, the spectral information is much richer data than a single frequency image. The problem is how to interpret and visualize multidimensional hyperspectral data. In this example, we show how SOM can be used for hyperspectral interpretation.

We set up the stage by discretizing the terrain into $n_1 \times n_2 = p$ pixels, numbering them, e.g., column by column. For each pixel, we have a radiance spectrum of n wavelengths, constituting the data vectors $x^{(j)} \in \mathbb{R}^n$, $1 \le j \le p$, stored in the columns of data matrix $\mathsf{X} \in \mathbb{R}^{n \times p}$. The attributes of this data are different wavelengths.

The coordinate lattice Q consists of equidistant lattice points in an equilateral triangle with side length k, as shown in Figure 6.7. Denoting the position vectors of the corners of the triangle by v_1, v_2 and v_3, each point q in the triangle can be expressed as

$$q = v_1 + \xi_2(v_2 - v_1) + \xi_3(v_3 - v_1) = \sum_{j=1}^{3} \xi_j v_j, \quad \xi_1 = 1 - \xi_2 - \xi_3.$$

The triplet (ξ_1, ξ_2, ξ_3) are known as the barycentric coordinates of the point. Observe that

$$0 \le \xi_j \le 1, \quad \xi_1 + \xi_2 + \xi_3 = 1,$$

therefore the barycentric coordinates provide a natural color coding, e.g., using the RGB color gamut, $(1, 0, 0)$ representing red, $(0, 1, 0)$ green, and $(0, 0, 1)$ blue. Using this RGB color coding, each point in the triangle corresponds to a color as shown in Figure 6.7.

Let K denote the number of lattice points, and let $m_\ell \in \mathbb{R}^n$ be the corresponding prototypes. At the end of the SOM algorithm the grid of prototypes have positioned themselves so as to approximate the data. For each pixel in the scenery, we find the nearest prototype to the data vector which, in turn, corresponds to a unique lattice point whose barycentric coordinates define a color. We assign this color to the pixel in consideration. Repeating the process pixel by pixel yields a color image of the scenery, where pixels far away from each other in the map but with similar spectra are assigned the same or similar color.

We test the idea using a hyperspectral data set "Indian Pines", collected by the AVIRIS infrared sensor flying above an agricultural area in northwestern Indiana. Each spectrum corresponds to 220 different wavelengths, ranging from 400 nm to 2 500 nm. The terrain consists of about two-thirds agricultural land and one third

Draft: do not distribute

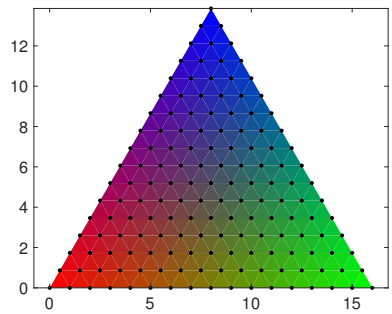

Figure 6.7. *The triangular coordinate lattice (black dots), and the RGB color coding of the triangle using the barycentric cordinates of each point as the RGB triplet.*

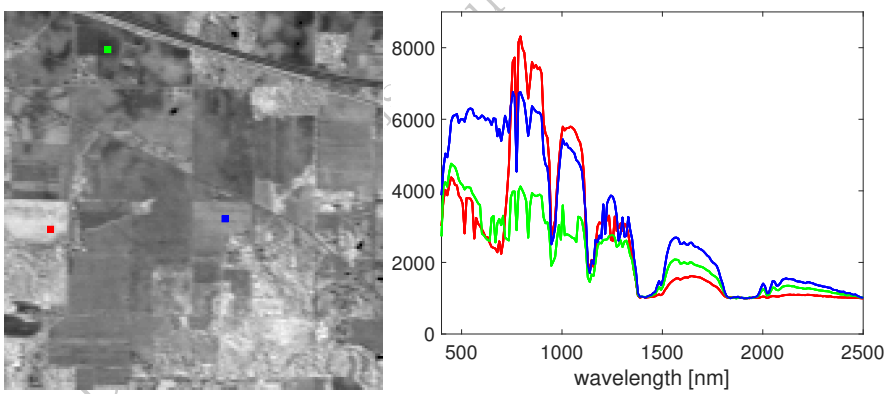

Figure 6.8. *A single wavelength black and white image. The selected wavelength was chosen arbitrarily as* $\lambda = 869.9\,\mathrm{nm}$. *Three representative pixels from the image are selected, and the corresponding spectra are plotted on the right, the color coding following the coloring of the pixel in the image on the left.*

forest, with a double highway, a railroad track, and some buildings. The data were collected in June 1992, when some of the soybean and corn were at an early stage of growth, and did not fully cover the fields. The left panel of Figure 6.8 shows a black-and-white image of the scenery, plotted by selecting arbitrarily one single wavelength, $\lambda = 869.9\,\mathrm{nm}$. To get an idea of the data, we have singled out three representative pixels, marked by a colored square in the image, and plotted the corresponding spectra in the right panel.

We run the SOM algorithm using the lattice in Figure 6.7 with $K = 153$ lattice points. The learning rate parameters α_0 and α_1 were chosen as in the previous example, and the coupling parameters were set to

$$\gamma_0 = 16/3, \quad \gamma_1 = 0.01,$$

where 16 represents the side length of the lattice triangle. The numbers of iterations

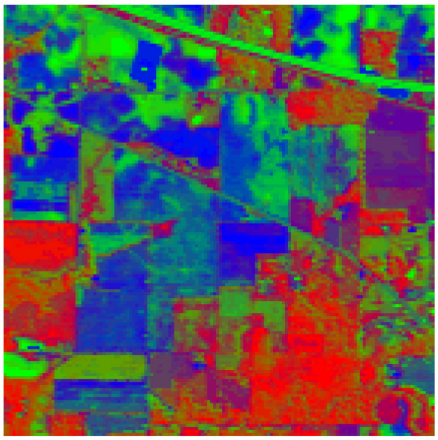

Figure 6.9. *The color image obtained by coding the color of each pixel according to the barycentric coordinates of the lattice point that corresponds to the nearest prototype.*

Draft: do not dis

were set to

$$T_0 = 5\,000, \quad T = 500 \times K = 76\,500.$$

The hyperspectral image interpreted with the SOM-induced coloring is shown in Figure 6.9. The image gives a quick summary of the scenery, using a color-based classification of its different parts. Sharp interfaces between differently colored patches are due to the fact that nearness if the pixels in the image implies no nearness of the corresponding spectra, and of their respective nearest prototypes.

Notes and comments

The SOM algorithm dates back to 1980's, and to the work of Teuvo Kohonen and his collaborators, see, [21, 22], or a more complete treatise, [35, 24, 25].

Approximating data in high-dimensional space using low-dimensional manifolds is a problem actively studied, and it is known as manifold learning [3]. The problem has a similarity with statistical regression. Another interesting approach is based on diffusion maps and harmonic analysis, see [6]. Both SOM and manifold learning are methods in a larger class, non-linear dimensionality reduction (NLDR) [28].

The Indian Pines hyperspectral data was downloaded from the site `http://lesun.weebly.com/ hyperspectral-data-set.html`, containing several other examples of hyperspectral images.